

A Report Into the Improvement of a Tabletop Gaming Web Application

Exam No. B024703

February 8, 2017

1 Introduction

This report describes a plan of action to improve a web application designed to assist the playing of a table top game. The code was received in a poor state, featuring errors in code that prevent it functioning properly, situations in which poor design decisions were taken by the original author, and situations where the specifications, as provided by the customer, have not yet been fully met.

The web application is written in Python and uses the Flask web framework.

In this report an approach will be taken that prioritises and improves upon failings in the current code in an agile iterative fashion, using aspects of the Scrum methodology. The aim is not simply to provide code that is more functional, but that is also extendible and maintainable for further work. Furthermore, as development time will be limited on this project, work will be scheduled so there will be frequent releases of shippable code, as opposed to the waterfall approach which might leave no publishable code when work stops.

The Scrum approach entails dividing the project into broad stories. These stories then have particular tasks associated with them. Work is undertaken in Sprints, short bursts of work in which prioritised tasks are completed from all or some of the broader stories. Sprint planning occurs before each individual sprint, allowing for a structured yet flexible approach, which allows

new issues to be dealt with as they arise in a sensible way. At the end of each sprint there should be some usable code, i.e. working code that has some way improved upon what was there before.

This report first describes at a high level problems with the current code and divides these into stories, before describing in more detail the specific issues or tasks related to each story. An outline of initial sprint plans showing the order and priority of will then be presented, followed by a Risk analysis and management strategy before a brief conclusion.

2 Problems and Stories

2.1 Developer Tooling and Documentation

At present there is no ReadMe file included with the source code describing how to run and do work to improve the code. The code is dependent on Python being installed on the host machine, as well as the Flask framework.

There is currently no source version control. This means that it would be hard to track changes during the development process. This would cause particular issues if there were more than one developer working on the code at once. In this case it would be essential that there was tooling in place that allowed many developers to work simultaneously on different issues in the code, tracking their changes individually and merging at the end of a task.

As well as tooling to track software changes, there should be tooling in place to track tasks being worked on within sprints. This tool should track which developers are working on which tasks, again preparing this project to be worked on by more than one developer.

In its current state there are no unit tests or continuous integration system in place for the project. Continuous integration would allow the code to be tested every time it is committed to the version control system. Unit tests are essential for maintainability, and for all future work tests should be written first following the 'Test-Driven Development' (TDD) [1] approach. These tests should be combined with a build system that ensures only thoroughly tested code gets presented as complete.

2.2 Code Formatting and Abstraction

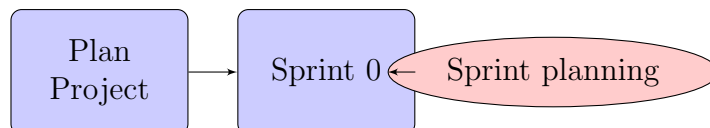
Almost all code for the project is, at present, contained within one source file. In this file are very long functions which each handle explicitly data storage, interfacing with the web framework and implementation game rules. All of these aspects should be decoupled from each other and broken into separate source files. This decoupling would make it much easier implement the unit tests described in 2.1 as well as undertake the further work described in the following stories.

2.3 Missing or Inaccurate Features from Game Specification

2.4 Data Store

3 Tasks

4 Work Plan



5 Risk Assessment

6 Conclusion

References

- [1] The Agile Alliance *TDD* <https://www.agilealliance.org/glossary/tdd/> Accessed: 01-02-2017